GRADING A CONTROL SYSTEM AGAINST REQUIREMENTS

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....5

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Measure the rise time, settling time, percentage overshoot and steady-state error of a closed-loop step response, by hand and with MATLAB.	☐
2	Judge whether a control system meets a set of written requirements, and identify which requirement each controller fails.	☐
3	Explain the trade-off between speed of response and overshoot, and select the controller that best meets the requirements.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder and the measurement of motor speed in revolutions per minute (RPM). The set-up is not repeated here.

The exercise uses the closed-loop model from the practical exercise, DC Motor Speed Control Using a PI Controller. The controller structure is not changed. Only the proportional gain K_p and integral gain K_i are changed between three pre-set values.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. The same speed step is applied to the motor under three different PI controllers. Each response is measured against a set of requirements, and the controller that best meets them is selected. The exercise supports the ACE-CORE learning guide, System Response and Requirements.

Background: Measures of the Step Response

Four measures are used to describe a closed-loop step response from an initial value y_0 to a final value y_∞:

- **Rise time** (t_r): the time taken for the response to go from 10% to 90% of the change.
- **Settling time** (t_s): the time after which the response stays within ±2% of the final value.
- **Percentage overshoot** (OS): how far the response goes past the final value, as a percentage of the change.
- **Steady-state error** (e_{ss}): the difference between the setpoint and the final value, as a percentage of the setpoint.

$$OS = \frac{y_{\max} - y_\infty}{y_\infty - y_0} \times 100\%, \qquad e_{ss} = \frac{r - y_\infty}{r} \times 100\%$$

A set of requirements gives a limit for each measure. A controller meets the requirements only if it meets every one of them. The requirements used in this exercise are given in Table 2. They are a starting point. Adjust them if your motor is noticeably faster or slower than the example motor.

Table 2: *Requirements for the DC Motor Speed Controller*

Measure	Requirement
Rise time, t_r	≤ 0.3 s
Settling time (2%), t_s	≤ 0.8 s
Percentage overshoot, OS	≤ 10%
Steady-state error, e_{ss}	≤ 2%

Run the Three Controllers

The three gain sets in Table 3 are chosen to give clearly different responses. Set A is cautious, set B is designed to meet the requirements, and set C is aggressive. The values suit the example motor and may need small changes for yours.

Table 3: *PI Gain Sets for the Three Controllers*

Controller	K_p (V/RPM)	K_i (V/(RPM·s))
A (cautious)	0.012	0.08
B (designed)	0.05	0.33
C (aggressive)	0.03	1.6

1. Open the PI speed control model and set the gains to controller A.
2. Apply a speed step from 0 RPM to 70 RPM at t = 0.5 s. Log the setpoint, the measured speed, the applied voltage and the time to the workspace.
3. Set the Stop Time to 3 s and run the model on the hardware.
4. Repeat for controllers B and C, saving the data from each run under a different name (for example *yA*, *yB* and *yC*).

Example results are shown in Figure 1. They are simulated with the motor model identified in the practical exercise, Parameter Estimation of a DC Motor using Linear Least Squares (K = 28 RPM/V, τ = 0.15 s), so they show the behaviour to expect rather than measured data.

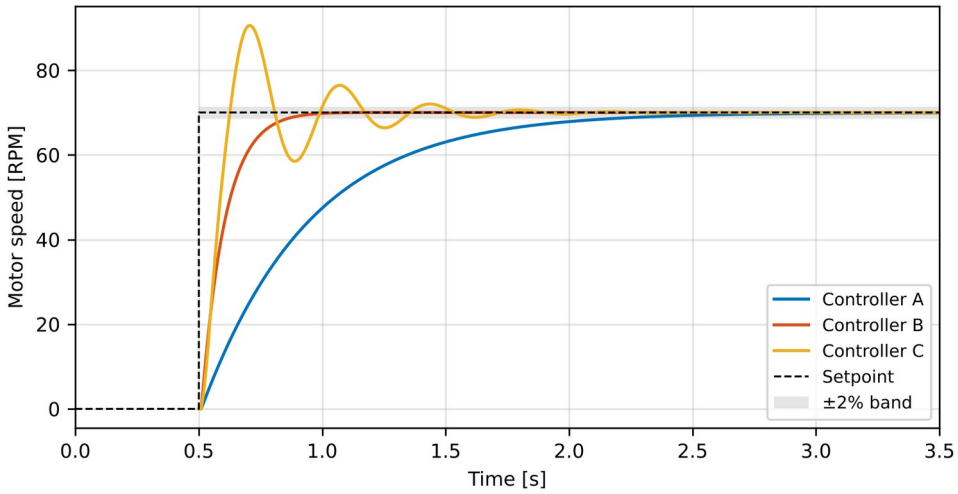


Figure 1: *Simulated Example of the Speed Response to a 70 RPM Step for Controllers A, B and C*

Measure the Responses

First measure each response by hand from the plot, then check your values with MATLAB. Measuring by hand first builds a feel for what each measure means. The script below uses the MATLAB function `stepinfo` with the logged data.

MATLAB Script: measuring a closed-loop step response against the requirements

```
%% Logged data for one controller (Structure With Time)
y_k = y.signals.values(:);          % measured speed [RPM]
t_k = t.signals.values(:);          % time [s]
r   = 70;  t_step = 0.5;            % setpoint [RPM] and step time [s]

idx = t_k >= t_step;
tn = t_k(idx) - t_step;  yn = y_k(idx);
y_inf = mean(yn(tn > tn(end)-0.5)); % final value [RPM]

S = stepinfo(yn, tn, y_inf, 0, 'RiseTimeLimits',[0.1 0.9], ...
             'SettlingTimeThreshold',0.02);
ess = abs(r - y_inf)/r*100;         % steady-state error [%]

fprintf('tr = %.2f s, ts = %.2f s, OS = %.1f %%, ess = %.1f %%\n', ...
        S.RiseTime, S.SettlingTime, S.Overshoot, ess);
```

Noise on the measured speed can make the settling time appear longer than it is, because noise spikes leave the ±2% band. If this happens, use the filtered speed from the practical exercise, Discrete-Time Low-Pass Filter Design for DC Motor Speed Measurement, or judge the settling time by eye from the plot.

Table 4: *Simulated Example Results for the A1.4 Example Model*

Controller	t_r (s)	t_s (s)	OS (%)	e_{ss} (%)	Meets all?
A	0.94	1.68	0.0	0.1	Fail
B	0.20	0.37	0.0	0.0	Pass
C	0.09	0.99	29.4	0.0	Fail

In the example, controller A is too slow and fails the rise time and settling time requirements. Controller C rises quickly but overshoots by almost 30% and then takes longer to settle. Only controller B meets every requirement. Record your own results in Table 5, marking each measure as a pass or a fail.

Table 5: *Results for Your Motor*

Controller	t_r (s)	t_s (s)	OS (%)	e_{ss} (%)	Meets all?
A					
B					
C					

Select a Controller

1. State which controller you would choose, and justify the choice in two or three sentences using your measured results.
2. Explain why the rise time and the overshoot tend to pull in opposite directions as the gains are increased.
3. Rewrite the requirements in Table 2 for a motor driving a conveyor belt that carries fragile items, and explain each change.

4.0 Concluding Remarks

This practical exercise measured the closed-loop step response of a DC motor speed controller against a set of written requirements. Using MATLAB, Simulink and ACE-Lab hardware, the rise time, settling time, percentage overshoot and steady-state error were measured by hand and checked with `stepinfo` for three different PI controllers.

The results showed that a controller must meet every requirement to be acceptable, and that the requirements often conflict: making a response faster tends to increase its overshoot. Requirements written for the application, such as limiting overshoot when the load is fragile, decide which controller is the right choice.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Grading a Control System against Requirements. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/XXXXXXXX